

RetryTaxBench: Evaluating Isolation Level Trade-offs under Bounded Retry on PostgreSQL

OLIVER ZHANG

SID: 520651281

Supervisor: Prof. Alan Fekete

School of Computer Science
The University of Sydney
Australia
15 April 2026

1. Introduction

Database management systems (DBMSs) underpin nearly every modern multi-user application, from banking to e-commerce. To keep data consistent when many users modify it simultaneously, these systems enforce isolation on transactions, sequences of operations treated as a single logical unit. Without adequate isolation, concurrent operations can produce anomalies such as write skew, where the final database state is corrupted despite each transaction being individually correct [7].

PostgreSQL, a widely used open-source relational DBMS, provides three primary isolation levels. The choice between these levels encodes a trade-off between speed and correctness. Read Committed is fastest and the default but permits several concurrency anomalies, whereas Serializable guarantees full correctness by detecting dangerous conflict patterns and aborting one of the conflicting transactions, forcing it to retry [2, 3].

This trade-off has motivated research into mixed-isolation allocation, where individual transaction templates within a workload are assigned the lowest isolation level that still collectively guarantees serializability [8]. Building on this theoretical foundation, Vandevort, Fekete et al. [1] proposed REPMILA (Read Promotion and Mixed Isolation Level Allocation), and showed on the SmallBank benchmark [5] that such allocations can significantly outperform standard SSI. Their experimental setup, however, assumed transactions retry indefinitely until they commit.

In realistic production environments, retrying forever is often impossible: transactions with external effects, such as charging a credit card, cannot safely be repeated many times, and most user-facing applications time out well before exhausting every possible retry. To investigate what changes when transactions cannot retry forever, we introduce RetryTaxBench, a custom Python harness built on the SmallBank benchmark and PostgreSQL that extends Vandevort et al.’s [1] methodology with a configurable per-transaction retry cap, and use it to evaluate whether the benefits of mixed isolation levels hold under bounded retry.

2. Background

2.1 The Role of Database Management Systems

A database management system (DBMS) manages a shared, persistent collection of data and mediates all access to it, so applications need not implement low-level storage, consistency, and access-control logic themselves [9]. By centralising storage and enforcing one consistent set of guarantees on behalf of every application, a DBMS eliminates the duplication, drift, and uncoordinated concurrent writes that plagued earlier file-based systems.

This report studies PostgreSQL, which is one of the most widely used open-source relational systems. Its source code and isolation-level internals are fully open to inspection, a practical advantage for a study that depends on understanding precisely how conflicts are detected and transactions aborted.

2.1.1 Transactions

A transaction packages one or more read and write operations into a single unit of work that the DBMS executes as an indivisible whole: either every operation takes effect, or none do [11, 12]. A bank transfer is the canonical example: withdrawing from one account and depositing into

another must succeed or fail together, since a partial transfer would lose or duplicate money. This all-or-nothing guarantee frees the application from handling every failure and concurrency scenario itself.

2.1.2 The ACID Model

Transactions are conventionally specified through four properties known by the acronym ACID [12]. Atomicity is the all-or-nothing guarantee just described; consistency requires a transaction to move the database only between states that satisfy the application's integrity rules; and durability requires that a committed transaction's effects survive any subsequent crash. This report concerns isolation: the guarantee that determines how much of one transaction's in-progress activity is visible to another running at the same time. The next two sections examine how PostgreSQL implements it, and the anomalies and trade-offs that follow.

2.2 Concurrency Control

Concurrency control is the mechanism by which a DBMS manages simultaneous access to data while preserving consistency. Its primary objective is serializability: a guarantee that the final database state is equivalent to some sequential, non-overlapping execution of all transactions. Running one transaction at a time provides this trivially but caps throughput at a single core, which is unacceptable for most production workloads. Modern systems instead interleave transactions across cores, but interleaving reintroduces the race conditions that serial execution avoided: the operations of one transaction may interfere with the logic of another.

2.2.1 Concurrency Bugs: Write Skew

Systems operating under guarantees weaker than serializability admit concurrency anomalies. A particularly subtle one is *write skew*: two concurrent transactions read the same situation and each write a different row, together violating an integrity constraint that neither violates on its own [15]. It differs from a lost update in that the two transactions never write to the same row, so a standard write-write conflict check never fires.

Consider two on-call doctors, Alice and Bob, covering a shift under a policy that at least one doctor remain on call [15]. Both want to go off call. Each transaction reads the on-call count, finds it is two, and concludes it is safe to leave because the other will remain; each then updates only its own row and commits. Because neither writes to the row the other reads, no conflict is detected and both commit, leaving zero doctors on call. The anomaly arises because each transaction's correctness rests on an assumption, that the other doctor stays on call, that was true when read but invalidated by a concurrent commit.

2.3 Isolation Levels in PostgreSQL

PostgreSQL offers three primary isolation levels, each occupying a different position on the trade-off between execution speed and the strength of guarantee provided against the anomalies described above [6].

2.3.1 Weak Isolation Levels

Most production applications default to weak isolation levels to maintain high throughput and low response times. PostgreSQL provides two:

- **READ COMMITTED (RC)**: the default PostgreSQL level. RC prevents transactions from observing uncommitted “dirty” data, but because it takes a fresh snapshot for every query rather than once per transaction, it remains vulnerable to read skew, where related rows updated partway through a transaction yield an inconsistent view.
- **REPEATABLE READ (Snapshot Isolation, SI)**: every operation in a transaction observes one consistent snapshot of the database, taken at the transaction’s start. This prevents most anomalies but remains vulnerable to write skew, since it does not reason about cross-record dependencies. To prevent lost updates, PostgreSQL enforces a *first-updater-wins* rule, aborting a transaction that modifies a row already changed by a concurrent one.

2.3.2 Serializable Isolation: From Locking to SSI (2005–2012)

For several decades, the standard way to guarantee full serializability was *two-phase locking* (2PL): transactions acquire shared locks for reads and exclusive locks for writes, and hold every lock until commit. This pessimistic approach prevents all anomalies, including write skew, but introduces lock thrashing, reduced concurrency, and frequent deadlocks, where two transactions wait on each other’s locks indefinitely.

A more practical alternative followed once Fekete et al. [13] proved that every non-serializable snapshot-isolation execution must contain two consecutive *rw-antidependencies* between concurrent transactions, a pattern they term a *dangerous structure* (the doctors example in Section 2.2.1 is one instance). Cahill et al. [2] used this to develop *Serializable Snapshot Isolation* (SSI): transactions run on an ordinary snapshot as under SI, but the system tracks rw-antidependencies with lightweight, non-blocking locks and aborts a transaction at commit time if doing so would complete a dangerous structure. This recovers the correctness of 2PL without its locking overhead, and was integrated into PostgreSQL as the **SERIALIZABLE** level in version 9.1 [3].

2.4 Adaptive Isolation Approaches

Hardware scaling has driven adaptive isolation models that move beyond assigning a single fixed isolation level to an entire workload, the fixed mix of transaction programs that an application repeatedly issues against the database.

2.4.1 The Multicore Scalability Crisis (2013–2023)

As hardware shifted toward higher core counts, SSI implementations were found to suffer a scalability collapse. Jung et al. [4] and Yu et al. [14] identified that internal kernel mutexes protecting shared serializability metadata become major points of synchronization contention, so that adding cores can paradoxically reduce throughput as the system spends more of its time managing internal locks than executing transactions. Han et al. [18] responded with *Physical-Logical Partitioning*, splitting the serializability-tracking structure to cut contentious critical sections by up to 85%. The same underlying problem was attacked from other directions in the following years, from optimistic concurrency control redesigned for thousand-core hardware [21] to contention-aware multi-versioned in-memory engines [22], and was still being revisited for modern storage engines as recently as 2023 [19].

2.4.2 Robustness Theory and REPMILA (2005–2025)

An older, complementary response avoids the multicore cost of full serializability for transactions that do not need it. Fekete [16] formalised this as *robustness*: a workload is robust against an isolation level exactly when no interleaving of its transactions under that level can complete a dangerous structure (Section 2.3.2), so every interleaving is serializable however the transactions overlap at runtime. Fekete [16] also gave a polynomial-time allocation algorithm, later complemented by automated robustness-testing techniques [17] that were subsequently generalised well beyond snapshot isolation [23, 24]. Robustness was then reformalised for Read Committed and an explicit transaction-template model [8, 20, 25], the journal version [8] being the formulation REPMILA builds on. *REPMILA* (Read Promotion and Mixed Isolation Level Allocation), proposed by Vandevoort et al. [1], automates the allocation of mixed isolation levels across a workload’s transaction programs, selectively promoting specific reads to writes to prevent dangerous interleavings while leaving the remaining transactions at a faster level. This study extends REPMILA by evaluating its reliability once the assumption of infinite retries is replaced by the bounded retry budgets available in practice.

3. Aims

REPMILA and other mixed-isolation schemes have so far been evaluated under retry-until-commit, where user-visible failure is zero by construction and the cost of aborts shows only indirectly, through throughput. As Section 1 noted, production systems rarely offer that guarantee: external side effects, client timeouts, and queueing delays all cap how many times a transaction can be retried.

This study aims to determine how a bounded retry budget changes the trade-off between throughput and reliability across PostgreSQL’s isolation levels, and specifically whether the throughput advantage that mixed isolation allocation holds over standard serializable isolation under unbounded retry persists once retries are limited. To this end, the study:

1. measures how retry budget, concurrency, and contention jointly affect throughput and the rate of user-visible failure under each isolation policy;
2. compares this behaviour across PostgreSQL’s standard isolation levels and a mixed-isolation allocation informed by robustness theory [1, 8]; and
3. assesses whether the conclusions drawn from Vandevoort et al.’s [1] unbounded-retry experiments still hold once that assumption is relaxed.

This study does not propose a new concurrency-control algorithm; its contribution is an empirical measurement of a cost that unbounded-retry evaluations cannot reveal.

4. Methods

This study measures the trade-offs in Section 3 by extending Vandevoort et al.’s [1] SmallBank/PostgreSQL setup with a configurable retry budget.

4.1 Experimental Setup

This study’s workload is SmallBank [5], a standard benchmark that models a simple banking application. Its schema has three tables:

1. Account(Name, CustomerID), mapping customer names to numeric IDs, and one-row-per-customer
2. Savings(CustomerID, Balance), and
3. Checking(CustomerID, Balance).

Five transaction programs operate over this schema:

1. Balance(N) reads a customer’s combined savings-and-checking balance;
2. DepositChecking(N, V) and TransactSavings(N, V) each add V to one of the two balances;
3. Amalgamate(N1, N2) zeroes both of N1’s balances and adds their total to N2’s checking balance; and
4. WriteCheck(N, V) deducts V from checking, deducting $V + 1$ instead if the combined balance cannot cover it, modelling an overdraft penalty.

In particular, WriteCheck is the transaction most prone to write skew (Section 2.2.1): two concurrent checks each evaluated against a stale, pre-deduction balance can both be approved even though the account cannot cover both.

The SmallBank configuration follows Vandervoort et al. [1] wherever feasible, varying only the parameters needed to test bounded retry (Table 1).

Table 1: Experimental setup compared with Vandervoort et al. [1]

Parameter	Vandervoort et al. [1]	This study
PostgreSQL version	16.2	16.2
Accounts	18,000	18,000
Hotspot size	20 accounts	20 accounts
Workload mix	Uniform over 5 transaction programs	Uniform over 5 transaction programs
Measurement window	60 s	60 s
Repeats per configuration	5	5
Retry model	Retry until commit	$N = 1, 3, 10, \infty$
Concurrency	100 clients	MPL = 1, 8, 32, 64
Hardware	Separate database and client machines	Co-located client and server

The hardware difference is a limitation rather than a design choice. Experiments run on a single 2021 MacBook Pro (Apple M1 Max, 10 cores, 64 GB RAM), with PostgreSQL and the client harness as separate processes on the same machine, whereas Vandervoort et al. [1] used a dedicated server and a separate client machine. Absolute throughput is therefore not comparable between the two studies; Section 5 instead focuses on the relative effect of retry budget, policy, concurrency, and contention within this study’s own controlled setup.

4.2 Isolation Policies

Four isolation policies are compared (Table 2). RC, SI, and SSI assign every program the same level; Mixed-robust applies the robustness-based allocation from Section 2.4.2, downgrading only DepositChecking to RC while the other four programs run at SSI.

Table 2: Isolation policies

Policy	Allocation	Role
RC	All five programs at Read Committed	Unsafe, high-throughput baseline
SI	All five programs at Repeatable Read	Snapshot Isolation baseline
SSI	All five programs at Serializable	Fully serializable baseline
Mixed-robust	DepositChecking at RC; Balance, TransactSavings, Amalgamate, WriteCheck at SSI	Vandevort et al.’s [1] no-promotion robust allocation

4.3 Retry Model and Harness

General-purpose benchmarking tools such as dbbench do not support assigning a different isolation level to each transaction type, or capping the number of retries, both of which this study needs. A custom Python harness was therefore built, following the structure of Vandevort et al.’s [1] reference implementation. It has four parts:

1. A configuration file listing the policy and parameters to test;
2. a controller that runs each test combination as a separate, restartable process;
3. a driver that resets the database and starts the worker processes for one run; and
4. a SmallBank module that sets PostgreSQL’s isolation level before each attempt (Algorithm 1).

For each logical request, the harness samples a transaction type and its account parameters once, before the first database attempt. If PostgreSQL aborts the transaction, the same transaction and parameters are retried under the same isolation policy, up to a retry budget N (Algorithm 1). $N = \infty$ reproduces Vandevort et al.’s [1] retry-until-commit assumption; finite N truncates the loop after N retries and records the request as a user-visible error.

Algorithm 1: Bounded-retry execution

```

procedure RUNWITHRETRY(conn, sampler, policy, N)
  tx, accounts, amount  $\leftarrow$  sampler.sample()  // sampled once
  isolation  $\leftarrow$  policy[tx]
  attempts  $\leftarrow$  0; aborts  $\leftarrow$  0
  while N = NONE or attempts  $\leq$  N do
    attempts  $\leftarrow$  attempts + 1

```

```

try
  set the session isolation level of conn to isolation
  execute transaction tx on conn with accounts, amount
  commit conn
  return COMMITTED(attempts, aborts)
except SERIALIZATIONFAILURE or DEADLOCK:
  rollback conn
  aborts  $\leftarrow$  aborts + 1
end while
return USERVISIBLEERROR(attempts, aborts)
end procedure

```

4.4 Test Configurations and Protocol

Every combination of the four isolation policies, four concurrency levels, four retry budgets and three contention levels described above was tested, each repeated 5 times, giving 960 runs in total. Each run followed the same steps:

1. Reset the database to a fixed starting snapshot; start the worker processes that send requests;
2. Run for 30 seconds without recording (warm-up);
3. Record the next 60 seconds of activity; then save the results.

The full set of 960 runs took approximately 24–27 hours to complete.

4.5 Metrics

Five metrics are recorded for each run (Table 3). Error rate is the metric of primary interest: under $N = \infty$ it is zero by construction, so any non-zero value observed under finite N directly measures the reliability cost that an unbounded-retry evaluation cannot reveal.

Table 3: Recorded metrics

Metric	Definition
Committed TPS	Committed transactions per measurement second
Attempted TPS	All database attempts per measurement second, including retries
Abort rate	$\text{Aborts} \div \text{total attempts}$
Error rate	$\text{Exhausted-retry failures} \div \text{logical requests}$
Latency	Time from first attempt to final commit or visible error

5. Results

This study ran 960 measurements in total, covering every combination of the four isolation policies, four concurrency levels, four retry budgets, and three contention levels described in

Table 4: Committed TPS under retry-until-commit ($N = \infty$) at hotspot probability $p = 0.9$.

Policy	MPL 1	MPL 8	MPL 32	MPL 64
RC	3728 ± 23	8582 ± 4361	9294 ± 3902	9705 ± 4074
SI	3564 ± 104	6953 ± 2699	5109 ± 1809	4483 ± 2035
SSI	3225 ± 558	6989 ± 3274	5151 ± 2324	5150 ± 2009
Mixed-robust	3641 ± 17	7205 ± 3429	5407 ± 2508	5661 ± 2650

Cells show mean $\pm$ 95% confidence interval over five repeats.

Section 4, each repeated five times. As expected, throughput only separates the four policies clearly at low concurrency, and error rate falls as the retry budget increases while rising with contention; abort rate, in contrast, stays persistently high throughout. Two results were not expected, and are discussed in turn below: SI fails and aborts more than SSI despite being the weaker isolation level, and Mixed-robust’s throughput advantage at low concurrency does not carry over into its failure or abort behaviour.

5.1 Throughput and concurrency

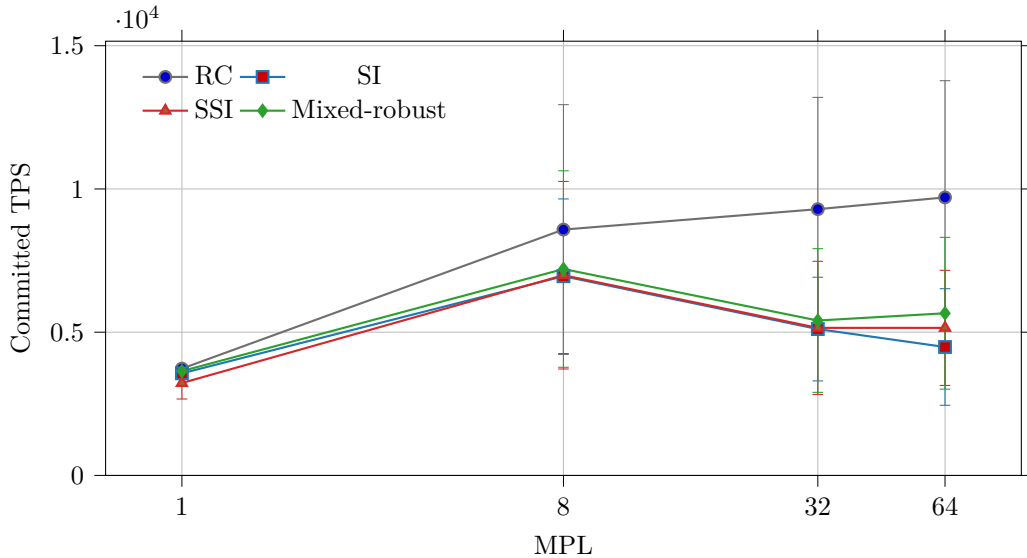


Figure 1: Committed throughput versus concurrency under retry-until-commit ($N = \infty$) at hotspot probability $p = 0.9$. Points are means over five repeats; whiskers show 95% confidence intervals. Throughput separates policies at MPL=1, but the intervals overlap at higher MPL.

Figure 1 and Table 4 show that throughput only ranks the four policies clearly at MPL = 1, where it follows isolation strength: RC and Mixed-robust are fastest, SI is slightly slower, and SSI is slowest. At higher concurrency, throughput has too high of variance to compare reliably, and the confidence intervals for all four policies overlap. Section 6 attributes this to the database and the client program sharing one machine’s CPU, so throughput is not used to compare policies elsewhere in this report.

5.2 Error rate under bounded retry

Table 5: User-visible error rate at MPL=64 and hotspot probability $p = 0.9$.

Policy	$N = 1$	$N = 3$	$N = 10$	$N = \infty$
RC	$0.00 \pm 0.00\%$	$0.00 \pm 0.00\%$	$0.00 \pm 0.00\%$	$0.00 \pm 0.00\%$
SI	$16.54 \pm 1.37\%$	$7.35 \pm 0.55\%$	$1.15 \pm 0.20\%$	$0.00 \pm 0.00\%$
SSI	$10.22 \pm 0.79\%$	$3.81 \pm 0.26\%$	$0.27 \pm 0.02\%$	$0.00 \pm 0.00\%$
Mixed-robust	$11.22 \pm 0.92\%$	$3.99 \pm 0.33\%$	$0.31 \pm 0.03\%$	$0.00 \pm 0.00\%$

Cells show mean $\pm$ 95% confidence interval over five repeats.

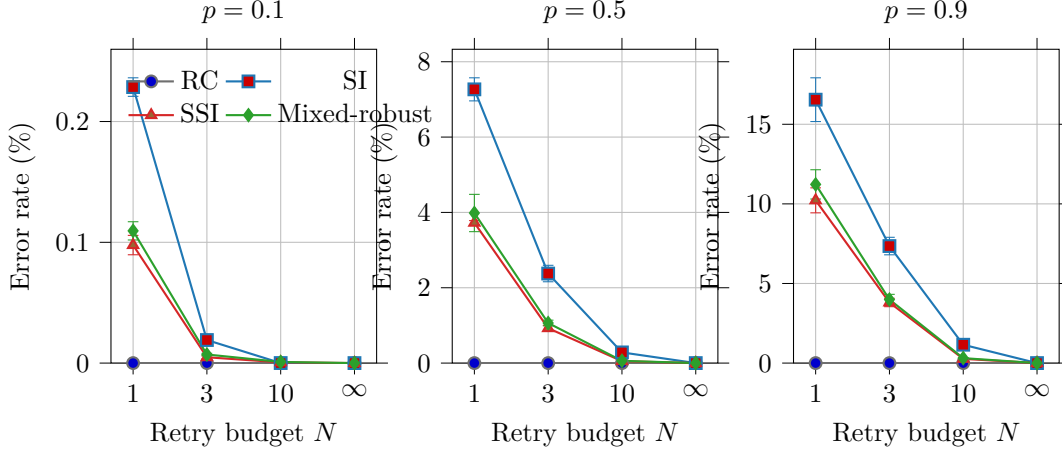


Figure 2: User-visible error rate versus retry budget at MPL=64. Each panel fixes hotspot probability p . Error rate falls with larger retry budgets and rises with contention. Points are means over five repeats; whiskers show 95% confidence intervals.

Figure 2 and Table 5 show that the error rate, the share of requests that exhausted their retry budget, falls as N increases and rises with contention, reaching 0% for every policy once retries are unbounded. Two results stand out: Mixed-robust tracks SSI closely rather than improving on it, and SI has a consistently higher error rate than SSI despite being the weaker isolation level. Both are discussed in Section 6.

5.3 Abort rate and latency: the cost that persists at $N = \infty$

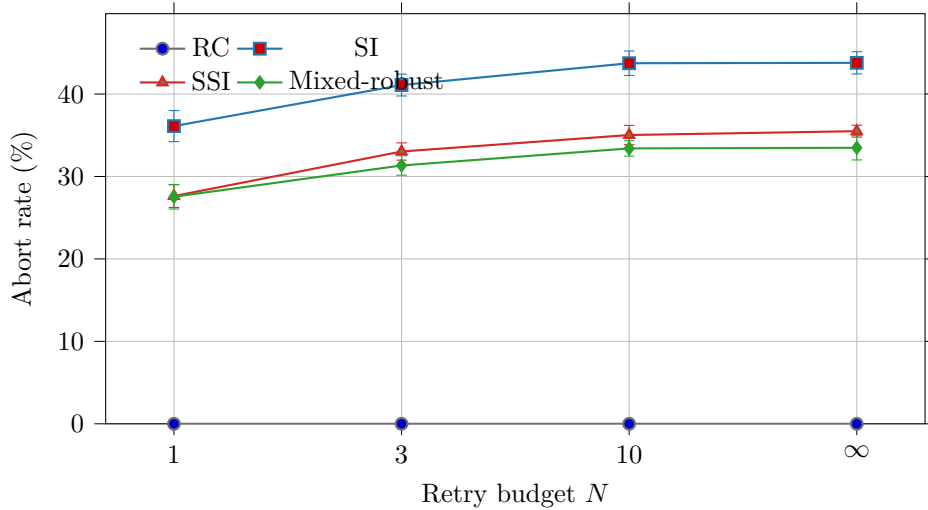


Figure 3: Abort rate versus retry budget at MPL=64 and hotspot probability $p = 0.9$. Larger retry budgets reduce user-visible failures but leave substantial abort pressure inside the database. Points are means over five repeats; whiskers show 95% confidence intervals.

Table 6: Abort rate and latency under retry-until-commit ($N = \infty$), MPL=64, hotspot probability $p = 0.9$.

Policy	Abort rate	Mean latency	p95 latency
RC	$0.00 \pm 0.00\%$	7.28 ± 3.22 ms	36.5 ± 23.2 ms
SI	$43.79 \pm 1.35\%$	15.39 ± 4.73 ms	83.7 ± 40.4 ms
SSI	$35.50 \pm 0.72\%$	13.33 ± 4.55 ms	61.3 ± 43.1 ms
Mixed-robust	$33.48 \pm 1.47\%$	12.57 ± 5.26 ms	66.5 ± 38.8 ms

Cells show mean $\pm$ 95% confidence interval over five repeats.

Figure 3 and Table 6 show that even when the error rate is 0% ($N = \infty$), the database is still aborting and re-running 33–44% of attempts internally, with the same $SI > SSI \gtrsim$ Mixed-robust ordering seen in Section 5.2. This is the central reliability cost this report measures: a workload that looks fully reliable under retry-until-commit is still discarding a large share of its work.

6. Discussion

The clearest result is that bounding retries changes how this reliability cost shows up, without making it go away. As the retry budget N increases from 1 to 10, SI’s failure rate falls sharply, from 16.54% to 1.15% (Table 5), but its abort rate over the same range rises slightly, from 36.11% to 43.74% (Figure 3). Allowing more retries hides the problem from the client without reducing how often the database itself has to abort and redo work. A retry-until-commit experiment, like Vandervoort et al.’s [1], would only see the first number and conclude the system is reliable.

This persistent 33–44% abort rate, present even when every request eventually succeeds ($N = \infty$), is not specific to PostgreSQL. Agrawal, Carey and Livny [26] showed that restart-based concurrency control, the family of techniques SI and SSI both belong to, only matches the throughput of older locking-based approaches while the system has spare capacity to absorb the work wasted by aborts; once contention removes that spare capacity, the wasted work dominates. Bounding the retry budget is what makes that wasted work visible as a client-facing error, rather than something only the database sees.

A more surprising result is that SI failed and aborted more often than SSI at every bounded N , even though SI is the weaker, faster isolation level and so would be expected to abort less often, not more. The gap was small at low concurrency but grew as concurrency increased, suggesting the cause lies in how PostgreSQL implements each level internally rather than in a fixed difference in how many conflicts each one detects. Confirming this would require breaking aborts down by cause, which Section 8 lists as future work.

Mixed-robust tracked SSI closely rather than beating it, with differences generally under two percentage points in both failure rate and abort rate. This matches Vandevort et al.’s [1] own finding for this allocation: little improvement over plain SSI as contention rises. Their suggested fix, rewriting specific transactions to avoid the conflict, was out of scope here. The underlying reason connects to a result from distributed systems: Bailis, Fekete et al. [28] show that an operation can skip coordination, and the aborts it causes, only if its correctness rule still holds when applied to two copies independently and merged. SmallBank’s rule that balances cannot go negative does not survive this for WriteCheck, which is why that transaction keeps paying the abort cost measured here.

Finally, the throughput results in Section 5.1 should be treated with caution. Because the database and the client program shared the same laptop, increasing concurrency increased competition for the same CPU cores, adding noise that grew with concurrency. This is the most likely reason no consistent throughput ordering between policies could be seen above $MPL = 1$. Running the client and the database on separate machines, as Vandevort et al. [1] did, would remove this confound (Section 8).

7. Conclusion

To conclude, this report’s contribution has been in the empirical study of the effect of bounding the number of retries to the REPMILA algorithm proposed by Vandevort 2025, and comparing its performance to known Isolation Policies. This project measured how a bounded retry budget reshapes the throughput–reliability trade-off across PostgreSQL’s isolation levels and a robustness-based mixed allocation (Aims 1-3). Throughput could not be measured reliably above $MPL=1$ on the co-located hardware used here, but error rate behaved predictably, falling with N and rising with contention (Aim 1), while abort rate remained persistently high regardless of N . Comparing policies produced one result matching Vandevort et al.’s [1] own prediction, Mixed-robust tracking rather than beating SSI (Aim 2), and one against the literature’s expectation, SI producing higher error and abort rates than the more conservative SSI. Most directly (Aim 3), bounding retries reveals what an unbounded-retry evaluation cannot: internal abort rates of 33-44% persist even where the bounded-retry error rate falls to zero, so a workload that looks fully reliable under retry-until-commit is still discarding a substantial share of its attempts.

8. Future Work

Four extensions follow directly from the results. First, repeating the experiment with the database and client on separate, dedicated machines, as Vandevort et al. [1] used, would remove the CPU-contention confound identified in Section 6 and allow throughput and latency to be reported with confidence. Second, splitting aborts by reason (deadlock, concurrent update, or serialization failure), already recorded by the harness but not yet analysed by category, would directly test whether PostgreSQL’s safe-retry and safe-snapshot optimizations explain why SI aborted more than SSI. Third, since the SI-SSI gap widens with MPL here, extending the same

comparison across the full range of MPL and hotspot-probability combinations, rather than the single MPL=64, p=0.9 case highlighted in this report, would test whether that trend continues at lower contention. Fourth, implementing Vandevoort et al.’s [1] read promotion on WriteCheck or Balance would test whether Mixed-robust can be made to outperform SSI rather than merely track it. Beyond these, testing retry budgets and isolation policies on other PostgreSQL versions and on transaction programs beyond SmallBank would clarify how far these findings generalise.

References

- [1] Vandevoort, B., Fekete, A., Ketsman, B., Neven, F., & Vansummeren, S. (2025). Using Read Promotion and Mixed Isolation Levels for Performant Yet Serializable Execution of Transaction Programs. *Proceedings of the VLDB Endowment*, 18(9), 2846–2858.
- [2] Cahill, M. J., Röhm, U., & Fekete, A. D. (2009). Serializable Isolation for Snapshot Databases. *ACM Transactions on Database Systems*, 34(4), 20:1–20:42.
- [3] Ports, D. R. K., & Grittnner, K. (2012). Serializable Snapshot Isolation in PostgreSQL. *Proceedings of the VLDB Endowment*, 5(12), 1850–1861.
- [4] Jung, H., Han, H., Fekete, A., Röhm, U., & Yeom, H. Y. (2013). Performance of Serializable Snapshot Isolation on Multicore Servers. In *Database Systems for Advanced Applications (DASFAA)*, pp. 51–65.
- [5] Alomari, M., Cahill, M., Fekete, A., & Röhm, U. (2008). The Cost of Serializability on Platforms That Use Snapshot Isolation. In *Proceedings of the 24th International Conference on Data Engineering (ICDE)*, pp. 576–585.
- [6] PostgreSQL Global Development Group. (2024). PostgreSQL Documentation: Transaction Isolation. Retrieved from <https://www.postgresql.org/docs/17/transaction-iso.html>
- [7] Berenson, H., Bernstein, P., Gray, J., Melton, J., O’Neil, E., & O’Neil, P. (1995). A Critique of ANSI SQL Isolation Levels. *ACM SIGMOD Record*, 24(2), 1–10.
- [8] Vandevoort, B., Ketsman, B., Neven, F., & Vansummeren, S. (2023). Robustness Against Read Committed for Transaction Templates. *Proceedings of the VLDB Endowment*, 16(11), 3372–3384.
- [9] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill.
- [10] Stonebraker, M., & Rowe, L. A. (1986). The Design of POSTGRES. *ACM SIGMOD Record*, 15(2), 340–355.
- [11] Gray, J. (1981). The Transaction Concept: Virtues and Limitations. In *Proceedings of the 7th International Conference on Very Large Data Bases (VLDB)*, pp. 144–154.
- [12] Haerder, T., & Reuter, A. (1983). Principles of Transaction-Oriented Database Recovery. *ACM Computing Surveys*, 15(4), 287–317.
- [13] Fekete, A., Liarokapis, D., O’Neil, E., O’Neil, P., & Shasha, D. (2005). Making Snapshot Isolation Serializable. *ACM Transactions on Database Systems*, 30(2), 492–528.
- [14] Yu, X., Bezerra, G., Pavlo, A., Devadas, S., & Stonebraker, M. (2014). Staring into the Abyss: An Evaluation of Concurrency Control with One Thousand Cores. *Proceedings of the VLDB Endowment*, 8(3), 209–220.

- [15] Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.
- [16] Fekete, A. (2005). Allocating Isolation Levels to Transactions. In *Proceedings of the 24th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*, pp. 206–215.
- [17] Jorwekar, S., Fekete, A., Ramamritham, K., & Sudarshan, S. (2007). Automating the Detection of Snapshot Isolation Anomalies. In *Proceedings of the 33rd International Conference on Very Large Data Bases (VLDB)*, pp. 1263–1274.
- [18] Han, H., Park, S., Jung, H., Fekete, A., Röhm, U., & Yeom, H. Y. (2014). Scalable Serializable Snapshot Isolation for Multicore Systems. In *Proceedings of the IEEE 30th International Conference on Data Engineering (ICDE)*, pp. 700–711.
- [19] Alhomssi, A., & Leis, V. (2023). Scalable and Robust Snapshot Isolation for High-Performance Storage Engines. *Proceedings of the VLDB Endowment*, 16(6), 1426–1438.
- [20] Vandevoort, B., Ketsman, B., Koch, C., & Neven, F. (2021). Robustness against Read Committed for Transaction Templates. *arXiv:2107.12239*.
- [21] Wang, T., & Kimura, H. (2016). Mostly-Optimistic Concurrency Control for Highly Contended Dynamic Workloads on a Thousand Cores. *Proceedings of the VLDB Endowment*, 10(2), 49–60.
- [22] Lim, H., Kaminsky, M., & Andersen, D. G. (2017). Cicada: Dependably Fast Multi-Core In-Memory Transactions. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*, pp. 21–35.
- [23] Nagar, K., & Jagannathan, S. (2018). Automated Detection of Serializability Violations Under Weak Consistency. In *Proceedings of the 29th International Conference on Concurrency Theory (CONCUR)*, LIPIcs 118, 41:1–41:18.
- [24] Beillahi, S. M., Bouajjani, A., & Enea, C. (2019). Checking Robustness Against Snapshot Isolation. In *Proceedings of the 31st International Conference on Computer Aided Verification (CAV)*, pp. 286–304.
- [25] Vandevoort, B., Ketsman, B., Koch, C., & Neven, F. (2022). Robustness Against Read Committed: A Free Transactional Lunch. In *Proceedings of the 41st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS)*, pp. 1–14.
- [26] Agrawal, R., Carey, M. J., & Livny, M. (1987). Concurrency Control Performance Modeling: Alternatives and Implications. *ACM Transactions on Database Systems*, 12(4), 609–654.
- [27] Gray, J., Helland, P., O’Neil, P., & Shasha, D. (1996). The Dangers of Replication and a Solution. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pp. 173–182.
- [28] Bailis, P., Fekete, A., Franklin, M. J., Ghodsi, A., Hellerstein, J. M., & Stoica, I. (2014). Coordination Avoidance in Database Systems. *Proceedings of the VLDB Endowment*, 8(3), 185–196.
- [29] Thomson, A., Diamond, T., Weng, S.-C., Ren, K., Shao, P., & Abadi, D. J. (2012). Calvin: Fast Distributed Transactions for Partitioned Database Systems. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, pp. 1–12.
- [30] Corbett, J. C., Dean, J., Epstein, M., Fikes, A., Frost, C., Furman, J. J., et al. (2013). Spanner: Google’s Globally-Distributed Database. *ACM Transactions on Computer Systems*, 31(3), 8:1–8:22.

[31] Abadi, D. J. (2012). Consistency Tradeoffs in Modern Distributed Database System Design. IEEE Computer, 45(2), 37–42.

Appendices

All the source code for this experiment can be found in the following Github repository:

- <https://github.com/ZhangO999/retry-tax-bench>

Personal Machine Stats:

14-inch Macbook Pro 2021, Apple M1 Max, 64GB Ram

MacOS Sequoia 15.7.4

10 Cores